SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

FACULTY OF LIBERAL ARTS AND BUSINESS STUDIES

DEPARTMENT OF COMPUTER APPLICATIONS

INTERNSHIP REPORT

NAME	:	MATHESHWARAN R
REGISTER NUMBER	:	RA2532241010311
CLASS	:	MCA SECTION: F
YEAR & SEMESTER	:	II YEAR / III SEMESTER
ORGANIZATION NAME	:	Department of Computer Applications, SRM Institute of Science and Technology (In-house Internship)
DOMAIN	:	WEB DEVELOPMENT INTERNSHIP
COURSE CODE	:	PCA25P01L
DATE	:	25 May 2026 – 25 August 2026
DURATION	:	3 Months

BONAFIDE CERTIFICATE

This is to certify that the Internship report titled “Web Development Internship on the CuriousBees Project at SRM Institute of Science and Technology” is a bonafide work carried out by MATHESHWARAN R (Register Number: RA2532241010311) for the award of the Degree of Master of Computer Applications. To my knowledge the work reported herein is the original work done by this student.

INTERNAL EXAMINER 1 **INTERNAL EXAMINER 2**

ACKNOWLEDGEMENT

I am grateful to the Department of Computer Applications, SRM Institute of Science and Technology, for giving me the chance to spend my summer building something real inside the university, and for trusting our team with the CuriousBees platform.

My sincere thanks to Dr P J Arul Leena Rose, Head of the Department, and Dr Albert Antony Raj S, Deputy Dean, for supporting this in-house internship. I owe a special word of thanks to my internship supervisor, Ms. Simi Gracia Sunil Christopher, Assistant Professor and Faculty Coordinator of the programme. Her guidance kept us focused whenever the project tried to grow in every direction at once, and her questions often made me rethink a design before it was too late to change it.

I thank the professors of the department for their encouragement and my academic mentor for the advice I received along the way. The ten teammates who worked with me on the AI/ML, Backend, Frontend and UI/UX, DevOps, Full Stack and Android tracks deserve much of the credit for what CuriousBees became, and leading them taught me as much as the code did. Finally, I thank my family and friends, who put up with long evenings and late deployments without a word of complaint.

MATHESHWARAN R

RA2532241010311

TABLE OF CONTENTS

INTERNSHIP COMPLETION CERTIFICATE 5
1. INTRODUCTION 6
1.1 Background of the Internship 6
1.2 Objectives of the Internship 7
1.3 Significance and Description of the Internship 7
1.4 Scope of the Report 8
1.5 Organization of the Report 8
2. COMPANY PROFILE 9
2.1 Overview of the Organization 9
2.2 Mission, Vision, and Values 10
2.3 Organizational Structure 10
2.4 Products or Services Offered 11
2.5 Internship Completion Certificate 12
3. ROLE AND RESPONSIBILITIES 13
3.1 Description of the Tasks and Projects 13
3.2 Tools and Technologies Used 15
3.3 Detailed Account of the Experience 16
3.4 System Architecture 18
3.5 API Design, Data Model and Access Control 19
3.6 Security and File Storage 21
3.7 Email-to-Event Extraction with AI 21
3.8 Testing, CI/CD and Cloud Deployment 23
4. DAILY/WEEKLY LOG OF ACTIVITIES 25
4.1 Weekly Log 25
4.2 Daily Log 27
4.3 Major Projects and Their Outcomes 28
4.4 Challenges Faced and How They Were Addressed 33
5. SKILLS ACQUIRED 35
5.1 Technical Skills 35
5.2 Soft Skills 36
5.3 New Knowledge and Competencies Developed 36
5.4 Analysis and Discussion 36
6. CONCLUSION 38
6.1 Summary of the Internship Experience 38
6.2 Influence of the Internship on Career Goals 38
REFERENCES 39
APPENDIX A: GLOSSARY OF TERMS 40

INTERNSHIP COMPLETION CERTIFICATE

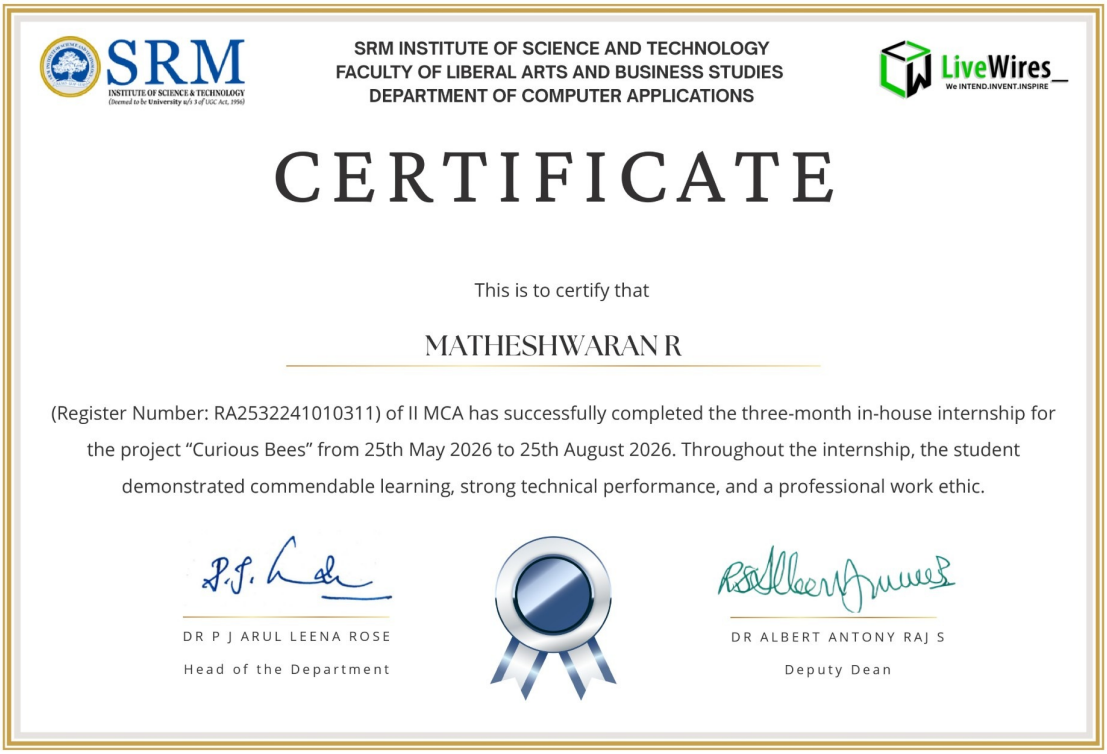


Figure 1.1: Internship Completion Certificate issued by the Department of Computer Applications, SRMIST

1. INTRODUCTION

1.1 Background of the Internship

Doctoral research at a large university involves a surprising amount of coordination. A scholar works with a supervisor for several years, submits progress reports, shares drafts and data, attends reviews and seminars, and looks for collaborators in other departments. Most of this happens over email threads, spreadsheets and messaging groups. When I first looked at how this worked in practice, the problem was not a lack of tools but too many of them. Information sat in different places, and announcements about events and opportunities reached only the people who happened to be on the right mailing list.

In 2026 the Department of Computer Applications of SRM Institute of Science and Technology (SRMIST) announced a Faculty-Guided Summer Internship Programme beginning on 25 May 2026. It was announced as a 30-day programme with a provision for extension, and its aim was to build a real-time collaborative platform under faculty guidance, with students organised into technical teams for UI/UX, frontend, backend, DevOps and AI/ML work. The programme was coordinated by Ms. Simi Gracia Sunil Christopher, Assistant Professor. It was later extended, and the completion certificate records a three-month in-house internship on the project "Curious Bees" from 25 May 2026 to 25 August 2026.

I joined as a second-year MCA student in my third semester, under course code PCA25P01L. I was made Team Lead of an eleven-member team and worked on the Full Stack – DevOps track myself. Our project was CuriousBees, a research collaboration and governance platform for SRMIST. It brings supervision, shared research workspaces, a research feed, opportunities, an events calendar and institutional administration into one web application for three groups of people: research scholars, research supervisors and institute administrators.

The project began on 29 May 2026 as a Phase 1 minimum viable product called SRM ReCollab, which was renamed ReCollab and finally CuriousBees. I built it first in my own repository and later moved the whole codebase, with its history, to the organisation repository we used for delivery. My teammates developed their modules in separate repositories, and bringing their work together became one of my main jobs. I continued after the internship period under the extension provision and delivered the final build on 4 October 2026. The production deployment will be carried out by the institution's ITKM team.

1.2 Objectives of the Internship

At the start of the programme I wrote down what I wanted to achieve. Looking back, the list held up well:

1. To build a full-stack web platform that the university could actually use, rather than a classroom prototype.
2. To understand how the parts of a modern web application fit together: the interface, the API, the database and the cloud services around them.
3. To model the institution faithfully, from campus to faculty to department, with three clearly separated user roles.
4. To integrate the services that research work depends on, such as sign-in, file storage, email and meetings.
5. To use AI technology where it removes real manual work, without letting it publish wrong information.
6. To practise secure development and automated testing from the beginning.
7. To prepare the platform for cloud deployment through a proper CI/CD pipeline.
8. To lead a team through planning, review and integration, and to document the system well enough for someone else to run it.

1.3 Significance and Description of the Internship

What made this internship different from coursework was that the software had a real owner. Accounts had to be tied to institutional email domains, the faculties and departments had to match the university's actual structure, research files had to stay private, and once we handed the platform over, a different team would have to deploy and maintain it. Each of these constraints forced a decision that a class assignment never asks for: which authentication provider to trust, how to change a database schema without losing data, and what the system should do when a configuration value is missing.

Figure 1.2 is a mind map of what the internship produced. On the left are the people who use CuriousBees and the work it supports; on the right are the technologies behind it. CuriousBees is a TypeScript monorepo. The web application is built with Next.js 15 and React 19 and has 75 page routes, and the API is built with NestJS 11 and exposes 253 HTTP endpoints across 40 controllers. Data lives in PostgreSQL, hosted on Supabase, and is accessed through Prisma with a schema of 54 models.



Figure 1.2: CuriousBees at a glance: users, research work, governance and the technology behind them

1.4 Scope of the Report

This report covers the internship period from 25 May to 25 August 2026, together with the integration and delivery work I did afterwards under the extension provision, up to the final build on 4 October 2026. It describes the host organisation, my role, the tools and ideas I worked with, a weekly and daily log, the problems we ran into and what I learned from them.

The figures in the report come from the delivered repository and its version-control history, and the screenshots come from the running application. No passwords, keys or other credentials appear anywhere in the report.

1.5 Organization of the Report

Chapter 2 profiles SRMIST, the department and our team. Chapter 3 explains my role, the tools I used and the main technical ideas behind the platform. Chapter 4 holds the weekly and daily logs, the outcomes with screenshots and the challenges we faced. Chapter 5 covers the skills I gained, and Chapter 6 closes with a summary and what the internship means for my career. The references and a short glossary follow at the end.

2. COMPANY PROFILE

Because this was an in-house internship, the host organisation was my own university. This chapter describes SRMIST and the Department of Computer Applications, and then the team I worked in.

2.1 Overview of the Organization

SRM Institute of Science and Technology is a deemed-to-be university under Section 3 of the UGC Act, 1956. The SRM group of institutions began with a primary school in 1968, and in 2002–2003 the Ministry of Education and the University Grants Commission approved SRMIST as a deemed university, with Dr T. R. Paarivendhar as its founder Chancellor. Today it offers diploma, undergraduate, postgraduate, doctoral and post-doctoral programmes in engineering and technology, medical and health sciences, science and humanities, management, law and agricultural sciences. Its strategic plan describes about 60,000 students and about 3,600 faculty members across five campuses. The university holds an A++ grade from the National Assessment and Accreditation Council (NAAC) and is placed in Category I by the Ministry of Education and the UGC (SRM Institute of Science and Technology [SRMIST], 2023). Its main campus is at Kattankulathur in Chengalpattu District, Tamil Nadu.

The Department of Computer Applications, part of the Faculty of Liberal Arts and Business Studies, has been running since 2006. It offers the BCA and BCA (Data Science) programmes at the undergraduate level and the MCA and M.Sc. Applied Data Science programmes at the postgraduate level (SRMIST, n.d.). Our own team had students from the MCA, MCA (Generative AI) and BCA programmes. The department ran the 2026 summer internship to give students hands-on, team-based experience on a real project. Table 2.1 summarises the details.

Table 2.1: Organization and Internship Details

Particular	Details
Organization	SRM Institute of Science and Technology (deemed to be university under Section 3 of the UGC Act, 1956)
Faculty	Faculty of Liberal Arts and Business Studies
Department	Department of Computer Applications
Campus	Kattankulathur, Chengalpattu District, Tamil Nadu
Internship type	In-house, faculty-guided internship
Project	CuriousBees (“Curious Bees”), a research collaboration and governance platform
My role	Team Lead and Full Stack Developer (Full Stack – DevOps track)

Particular	Details
Internship period	25 May 2026 to 25 August 2026 (3 months)
Faculty Coordinator	Ms. Simi Gracia Sunil Christopher, Assistant Professor
Certificate signatories	Dr P J Arul Leena Rose, Head of the Department; Dr Albert Antony Raj S, Deputy Dean
Final delivery	4 October 2026, for production deployment by ITKM
Website	www.srmist.edu.in

2.2 Mission, Vision, and Values

The university's vision is to become a world-class university that creates and shares knowledge, and to give its students a distinctive learning experience in science, technology, medicine, management and other fields that serves the wider world (SRMIST, 2023). Its mission has three parts: to move up through international alliances and collaboration towards global excellence; to advance knowledge in a rigorous academic and research environment; and to attract and develop people in an environment that encourages freedom, creativity and innovation.

The core values in the strategic plan are excellence, integrity and academic freedom; a global vision with local commitment; and inclusiveness, diversity and empathy (SRMIST, 2023). The university's emblem carries the motto "Learn · Leap · Lead".

Some of these values turned into very practical rules on our project. Research data had to stay private to the people working on it, every sensitive administrative action had to leave an audit trail, and the platform should never show an event or a fact it could not stand behind. That last rule shaped how we used AI later on.

2.3 Organizational Structure

The internship ran as a faculty-guided team inside the department (Figure 2.1). The Faculty Coordinator guided the project and reviewed our progress. As Team Lead I sat between her and the six technical tracks: I planned the work, divided it among the tracks, reviewed what came back and integrated it into the delivered product, while also writing a large part of the code myself. Table 2.2 lists the members of the team.

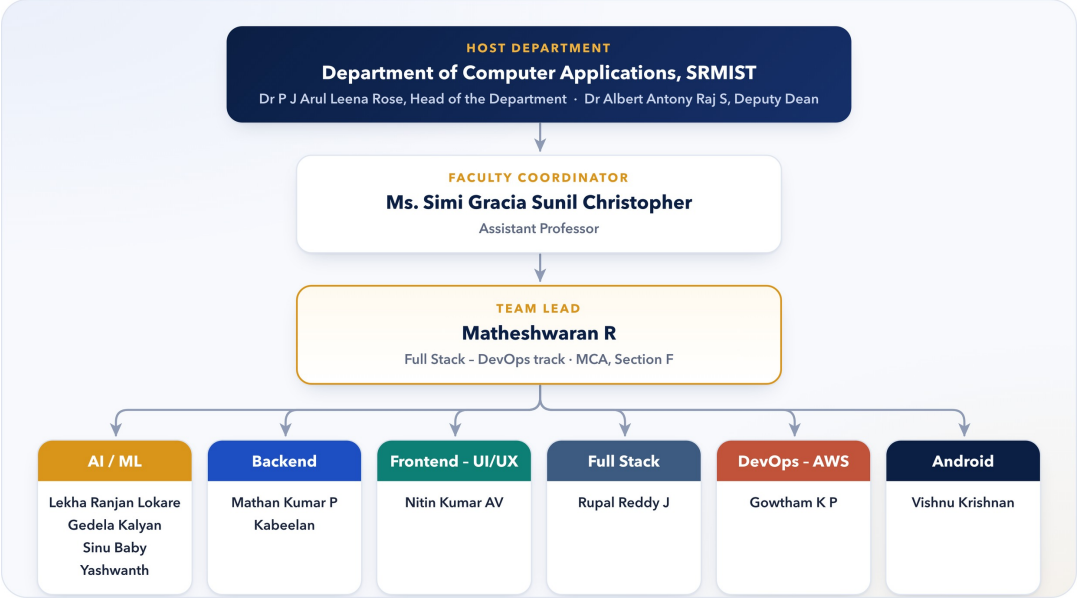


Figure 2.1: Organizational structure of the CuriousBees project team

Table 2.2: CuriousBees Project Team

S. No.	Name	Programme	Section	Team / Track
1	Lekha Ranjan Lokare	MCA (Generative AI)	D	AI/ML
2	Gedela Kalyan	MCA (Generative AI)	D	AI/ML
3	Sinu Baby	MCA (Generative AI)	B	AI/ML
4	Yashwanth	MCA (Generative AI)	D	AI/ML
5	Vishnu Krishnan	BCA	A	Android
6	Mathan Kumar P	MCA (Generative AI)	B	Backend
7	Kabeelan	MCA	B	Backend
8	Gowtham K P	MCA (Generative AI)	B	DevOps – AWS
9	Nitin Kumar AV	BCA	A	Frontend – UI/UX
10	Rupal Reddy J	MCA (Generative AI)	B	Full Stack
11	Matheshwaran R	MCA	F	Full Stack – DevOps (Team Lead)

The tracks matched the teams named in the internship announcement, with an extra Android track. Seeing a team's work from the lead's side for the first time, I learned quickly how much depends on agreeing on interfaces early.

2.4 Products or Services Offered

As a university, SRMIST offers diploma, undergraduate, postgraduate, doctoral and post-doctoral programmes and supports research in each discipline (SRMIST, 2023). The Department of Computer Applications contributes undergraduate and postgraduate programmes in computer applications and data science (SRMIST, n.d.).

What our team delivered to the institution was the CuriousBees platform, and Table 2.3 shows what it offers each role. One design choice needs a word of explanation: institute administrators deliberately do not take part in research. A guard in the API called ResearchParticipantGuard keeps research features for scholars and supervisors, while administrators look after accounts, institutional structure, moderation and auditing. Keeping governance and research apart made the permission rules much easier to reason about.

Table 2.3: Capabilities of CuriousBees by User Role

Role	What the platform provides
Research Scholar	• Onboarding with faculty and department selection • Supervisor requests within the same department (up to six scholars per supervisor) • My Research: thesis topic, research stage, milestones and progress reports • Workspaces with files, milestones, updates and meetings • Research feed, comments, follows and collaboration requests in Curious Nexus • Opportunities, the events calendar and notifications
Research Supervisor	• Supervision Panel to accept or decline requests and follow scholars • Review of progress reports • Creation and ownership of research workspaces • Posting of opportunities and calendar events
Institute Administrator	• Accounts created by administrators, singly or in bulk from a spreadsheet • Faculties, departments and campuses; roles and permissions • Moderation, announcements and institutional events • Event Inbox for events read from email • Email statistics, analytics, audit centre and system settings

2.5 Internship Completion Certificate

At the end of the internship the Department of Computer Applications issued me an Internship Completion Certificate, reproduced as Figure 1.1. It confirms that I, Matheshwaran R (Register Number RA2532241010311) of II MCA, completed a three-month in-house internship on the project “Curious Bees” from 25 May 2026 to 25 August 2026, and it records that I showed commendable learning, strong technical performance and a professional work ethic. The certificate is signed by Dr P J Arul Leena Rose, Head of the Department, and Dr Albert Antony Raj S, Deputy Dean, and carries the SRM and LiveWires emblems with the tagline “We Intend. Invent. Inspire.” It meant a lot to me, mostly because it recognised work on something that people at the university will actually use.

3. ROLE AND RESPONSIBILITIES

3.1 Description of the Tasks and Projects

My role had two halves that were hard to separate: leading an eleven-member team, and building large parts of the platform myself. In a typical week I might plan the next phase with the Faculty Coordinator, review a teammate's backend module, chase down a deployment problem and write a new feature, sometimes all on the same day. Figure 3.1 shows the cycle that every module went through.

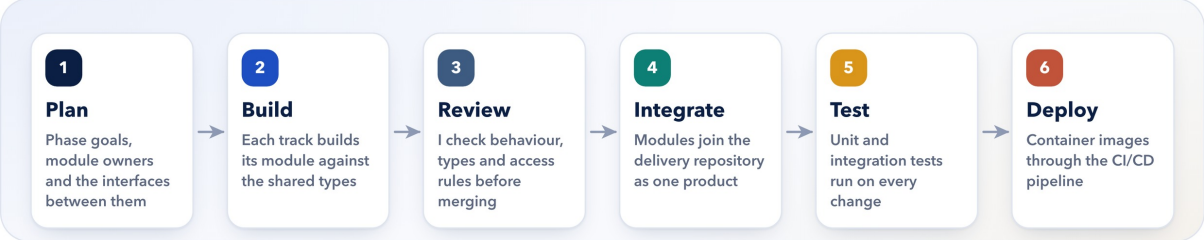


Figure 3.1: The cycle each module went through, from planning to deployment

Each step depended on the one before it. A module that was not reviewed properly was harder to integrate, and an integrated system without automated tests could break in one place because of a change somewhere else. I learned to protect the review and test steps even when the deadline pushed the other way.

3.1.1 Leading the Team

I split the project into phases: a Phase 1 minimum viable product in the first week, then consolidation of the architecture and the main portals, then feature work, a Phase 2 migration to a proper institutional hierarchy, and finally production readiness. Each phase was written up in the repository so that everyone could see what "done" meant.

I assigned modules to the AI/ML, Backend, Frontend and UI/UX, DevOps, Full Stack and Android tracks according to what each person was strongest at. Two shared packages, @curiousbees/types and @curiousbees/constants, acted as the contract between the frontend and the backend, so people could build in parallel without stepping on each other. Because my teammates worked in their own repositories, integration was where most of the real difficulty showed up: two modules that each worked perfectly on their own could disagree about a data shape or an access rule. Resolving those conflicts, and explaining why I chose one approach over another, was a large part of my job. I also kept the documentation going; by delivery the repository had a README and 40 documents covering the architecture, database, deployment, security, the design system, setup guides for macOS, Linux and Windows, and a release checklist.

3.1.2 Building the Platform

I designed the monorepo and wrote much of both applications. On the backend, I organised the NestJS API into feature modules, each with its own controller, service and data-transfer objects. Table 3.1 lists the main ones.

Table 3.1: Principal API Modules and Their Responsibilities

Module	Responsibility
auth, users, onboarding	Session checks, the current user's profile, onboarding of scholars and supervisors
admin	Single and bulk account provisioning, roles, user status, institution settings, safety rules
faculties, departments, research-domains	The institutional hierarchy and the research taxonomy
supervisor-requests, supervisors	Supervision requests, approvals, capacity limits and reassignment
my-research, reports, publications	The research record, progress reports and publications
workspaces, files	Shared workspaces, members, milestones and updates; S3 uploads and downloads
threads, comments, feed, collaborations	Research feed, nested comments, likes, saves, shares and collaboration requests
opportunities, events, announcements	Opportunities, the events calendar and system announcements
notifications, search	In-app alerts, Brevo email, the daily digest; search with a PostgreSQL fallback
integrations, event-ingestion	Google Workspace meetings and Chat spaces; the email-to-event workflow and its review queue

A few of the features I enjoyed building most:

- Supervision. A scholar requests a supervisor in their own department, and the API refuses the request if that supervisor already has six scholars. Brevo emails let both sides know when a request is sent, accepted or declined.
- Workspaces and files. Uploads of up to 50 MB go straight from the browser to a private S3 bucket through presigned URLs that expire after five minutes, so large files never pass through our API.
- Notifications. Alongside in-app alerts and email, a daily 8:00 AM event digest takes a PostgreSQL advisory lock, so it runs exactly once even when several API instances are running.

- The governance console, with user provisioning from a spreadsheet, faculties and departments, moderation, announcements, email statistics, analytics, the audit centre and the Event Inbox.
- The frontend structure: role-based portals in the Next.js App Router and many of the shared components. Near the end I led a redesign that introduced a documented design system on Tailwind CSS with light and dark themes and reduced-motion support (Tailwind Labs, n.d.).

3.1.3 AI and Automation

SRMIST announces most of its events by email, and I wanted those events to appear on the CuriousBees calendar without anyone retyping them. With the AI/ML track I built a pipeline in which an n8n workflow reads a dedicated mailbox, filters the messages with fixed rules, asks the Gemini 2.5 Flash model to extract the event details in a strict format, and sends the result to our API, which decides whether to publish it. Section 3.7 explains how it works.

Our first attempt looked quite different. In the first week we had a prototype that used a local large language model through Ollama, and an analytics service that grouped events by meaning with pgvector embeddings. Both were removed during the restructuring on 4 June. They were interesting to build, but a hosted model behind strict validation turned out to be simpler to run and far more predictable.

3.1.4 Cloud and DevOps

I prepared the platform for deployment more than once as it grew: as serverless functions on Vercel in the first week, on Railway in June, and finally as a complete design for Amazon Web Services (AWS), with container images, infrastructure templates, least-privilege access policies and a CI/CD pipeline in GitHub Actions (Section 3.8). I also wrote the automated test suites and the safeguards that keep tests away from real data. In October 2026 I moved all of the project's development and demonstration data from local machines into the shared Supabase database, so the whole team now works against one cloud source of truth.

3.2 Tools and Technologies Used

I used TypeScript everywhere, so that the web application, the API, the scripts and the tests could share the same language, types and tooling (Microsoft, n.d.). Table 3.2 lists the main tools with the versions recorded in the project at delivery. For almost all of them, the official documentation listed in the References was where I did my learning.

Table 3.2: Tools and Technologies Used in the CuriousBees Project

Category	Technology (version)	Purpose in CuriousBees
Language and runtime	TypeScript 5.9; Node.js 22	One language for frontend, backend, scripts and tests
Frontend	Next.js 15.5 (App Router); React 19.2	Role-based portals and 75 page routes
UI and client state	Tailwind CSS 3.4; Framer Motion 11; Zustand 4.5; TanStack Query 5; React Hook Form 7; Zod 3	Design system, themes, application state and validated forms
Backend	NestJS 11.1 on Express; Helmet; Throttler; class-validator; Winston	Modular REST API, security headers, rate limits, validation and logs
Database	PostgreSQL on Supabase; Prisma 6.19	A 54-model relational schema and its migrations
Authentication	Supabase Auth with Google sign-in	Institutional sign-in (earlier Firebase, then Clerk)
Files, email and meetings	Amazon S3 presigned URLs; Brevo; Google Workspace	Private files, transactional email, Meet links and Chat spaces
Search	Amazon OpenSearch (optional)	Full-text search with a PostgreSQL fallback
AI and automation	n8n; Gemini 2.5 Flash; Gmail API (early prototype: Ollama, pgvector)	Email-to-event extraction with structured output
Testing	Jest 30; Supertest 7; Node.js test runner	Unit, integration and workflow tests
Containers and CI/CD	Docker; Docker Compose; GitHub Actions	Repeatable builds and automated checks on every change
Cloud	AWS ECS Fargate, ECR, CloudFront, S3, IAM (earlier Vercel and Railway)	The production deployment prepared for ITKM
Version control	Git and GitHub; npm workspaces	Monorepo with 129 commits up to delivery

Each external service had one clear job. Supabase proves who a user is, S3 stores files, Brevo sends email (Brevo, n.d.) and Google Workspace creates meetings and Chat spaces (Google for Developers, n.d.). I kept each of them behind a small module in the API, so any one of them could be switched off in development or replaced later without touching the rest of the code. That paid off more than once. During the internship we also had a Zoom meeting provider; we have since retired it so that meetings run through Google Workspace alone, and because it lived in its own module, taking it out did not disturb anything else.

3.3 Detailed Account of the Experience

The internship started on 25 May 2026, and my first goal was simple: give the team something that ran. By 29 May I had a monorepo with a Next.js web application, a NestJS API and shared

TypeScript packages, connected to a Supabase PostgreSQL database and deployed on Vercel. Getting that deployment to work took far longer than writing the code. A Prisma-based NestJS API does not run as a serverless function out of the box, and it took eight deployment commits over two days, adjusting the Prisma binary targets, the routing and the API prefix, before it finally stayed up. By the end of the week we had a deployable product with sign-in and a first AI prototype.

The second week was the busiest of the internship, with 44 commits. Seeing the first version running made its weaknesses obvious, so I restructured the code into feature modules. That let us split the work across the tracks and build workspaces, the opportunities portal, the admin dashboard and onboarding in parallel. It was also the week I learned something about scope. I added Redis and BullMQ queues for notifications, checked how they would behave in deployment, and took them out again within days, because they meant one more service to run for very little benefit at our scale. In the same week I moved sign-in from Firebase to Clerk and hardened the API for Railway.

Over the third to fifth weeks the governance console and the social side of the platform took shape: institute administration, configurable allowed email domains, accessible public pages and a research feed with discovery, nested comments, saved posts, sharing and sorting.

From the end of June to early August only one change landed in the delivery repository, a fix on 13 July to the API's network binding. This was the period in which my teammates were building their modules in their own repositories. Work in the main repository picked up again on 10 August. In the last two weeks of the internship I built the events and opportunities dashboards, the Curious Nexus hub, the researcher directory, multi-level comments and collaboration requests. I also decided to move authentication to Supabase, added the integrations module, and built supervision request management with reassignment and a standard limit of six scholars per supervisor. On the last day I removed the socket-based chat module in favour of something simpler. That pattern repeated throughout the project: once the real requirements were clear, the right answer was usually fewer moving parts.

After 25 August I continued under the extension provision. The final stretch, from 28 September to 4 October, was all about delivery: the Phase 2 migration to a canonical institutional hierarchy, the AWS infrastructure and CI/CD pipeline, administrator safety rules and protection for the root administrator, the platform redesign, single and bulk user provisioning, Google Workspace Chat spaces for workspaces, and the email-to-event pipeline

with its review queue. Alongside all of this I was merging my teammates' modules and writing the documentation ITKM will need to deploy the platform.

3.4 System Architecture

A web application is usually split into three tiers: the presentation tier in the browser, the application tier on a server, and the data tier where information is stored. Figure 3.2 shows how CuriousBees follows this pattern. The Next.js application, built on React (Meta Open Source, n.d.; Vercel, n.d.), is the presentation tier; the NestJS API is the application tier; and PostgreSQL on Supabase is the data tier. Keeping them apart meant each could be built, tested and secured on its own.

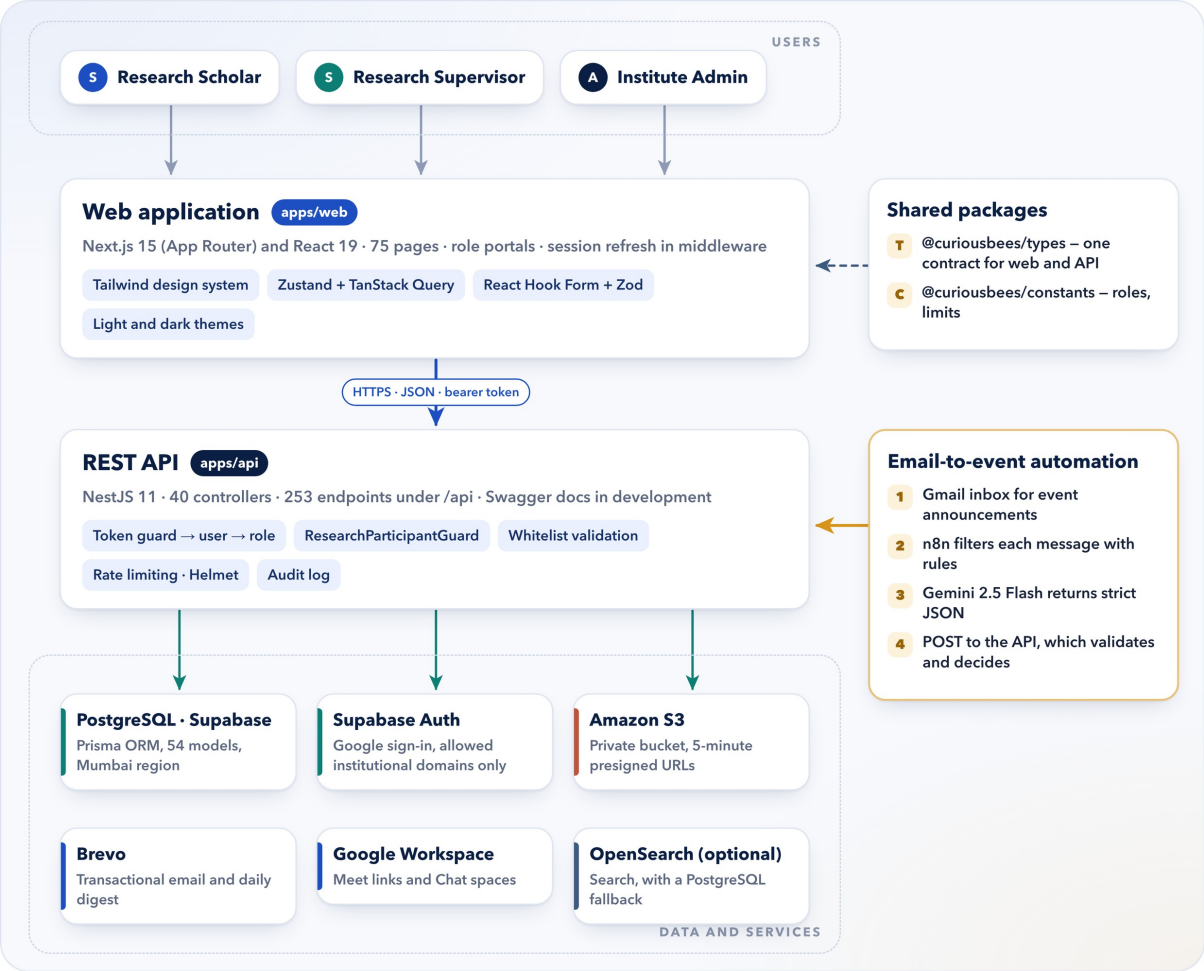


Figure 3.2: High-level architecture of CuriousBees

Three decisions in this picture shaped almost everything else. First, the web application never talks to the database directly. Every read and write goes through the API, which is the only place where permissions are checked; route protection in the browser exists only to send people to the right portal. Second, both applications import the same type definitions from a shared package, so when a data shape changes, the TypeScript compiler points out every place that

needs updating before anything reaches a browser. With several people building at once, this caught mistakes that would otherwise have surfaced only during integration. Third, the AI workflow is simply another client of the API, with its own machine token, and it cannot do anything a normal request could not.

3.5 API Design, Data Model and Access Control

REST is an architectural style in which a server exposes resources and clients act on them through a uniform interface (Fielding, 2000). In CuriousBees each resource has a URL, and the HTTP method says what should happen: GET reads, POST creates, PATCH updates and DELETE removes. NestJS groups each feature into a module with a controller that maps routes and a service that holds the business rules (NestJS, n.d.). A global validation pipe accepts only the fields each request declares, so a browser cannot quietly set its own role, and lists that can grow, such as the feed, use cursor-based pagination capped at 50 items per page.

The data model was the part I rewrote most often. Prisma describes the tables of a PostgreSQL database in one schema file and generates a typed client from it (Prisma Data, n.d.; PostgreSQL Global Development Group, n.d.). Figure 3.3 shows the core entities. In the early versions a user's department was free text, so "Computer Applications", "Dept. of Computer Applications" and "MCA Department" counted as three different departments. The Phase 2 migration replaced the text with a reference to a single Department record, which belongs to a Faculty and a Campus. After that, a question such as "which supervisors are in this department?" finally had one correct answer, and renaming a department changed it everywhere at once (Kleppmann, 2017).

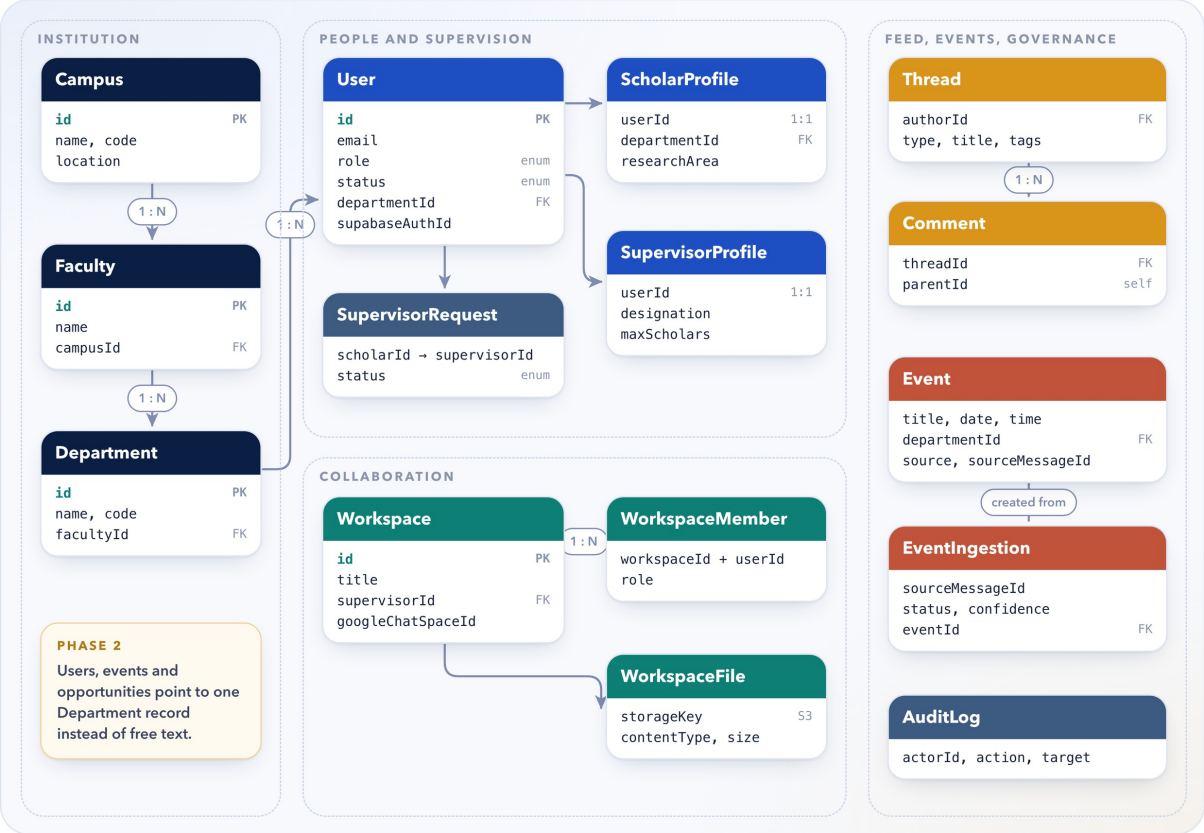


Figure 3.3: Core entities of the CuriousBees data model (simplified)

Every schema change after the production baseline is a forward-only migration applied with prisma migrate deploy, which never drops data, and the CI pipeline builds a fresh database from the migrations to check that it matches the schema exactly.

Authentication and authorization are kept apart on purpose. Supabase Auth answers the question “who is this person?” (Supabase, n.d.), and the API answers “what may they do?” using only its own database. Accounts are created by administrators, sign-in is limited to allowed institutional domains, and anything the browser claims about roles is ignored. Because of this separation we were able to change the authentication provider twice without rewriting a single access rule. Table 3.3 summarises the main permissions.

Table 3.3: Role-Based Access Control in CuriousBees

Action	Scholar	Supervisor	Admin
Sign in with an allowed domain and an administrator-created account	Yes	Yes	Yes
Request a supervisor; submit progress reports	Yes	—	—
Accept or decline supervision requests; review reports	—	Yes	—
Create research workspaces and post opportunities	Yes	Yes	—
Post, comment and follow in the research feed	Yes	Yes	—
Post events on the calendar	—	Yes	Yes

Action	Scholar	Supervisor	Admin
Provision accounts; manage faculties and departments	—	—	Yes
Moderate content; view the audit centre, analytics and Event Inbox	—	—	Yes

3.6 Security and File Storage

I used the OWASP Top 10 as a checklist while building (OWASP Foundation, 2021). The most important item, broken access control, is handled by checking every permission on the server with role guards, the ResearchParticipantGuard and workspace membership checks. Database queries go through Prisma, so they are parameterised. Helmet adds security headers, cross-origin calls are limited to an allow-list, rate limits protect the sign-in and API routes, and the API refuses to start in production if a required setting is missing or unsafe. Sensitive administrative actions are written to an audit log.

Files needed special care. Thesis drafts and datasets must stay private to a workspace, and passing large files through the API would waste its memory. CuriousBees uses presigned URLs instead: short-lived links, signed by the server, that let the browser upload or download one specific object directly from a private S3 bucket (Amazon Web Services, n.d.). Before issuing a link, the API checks that the user belongs to the workspace, allows only documents, spreadsheets, images, archives and text files, rejects executable extensions anywhere in the name, and signs the 50 MB limit into the URL so that S3 itself refuses anything larger. Each link expires after five minutes, and after an upload the API checks the stored object's real size and type and deletes it if they do not match.

3.7 Email-to-Event Extraction with AI

This was the part of the project I found most interesting, because it made me think hard about where AI helps and where it can do damage. A large language model can read an announcement email and pull out the title, date, time, venue and links. It can also misread a date, invent a link that was never in the email, or mistake a newsletter for an event, and sending every email to a paid model would cost money for no reason. The pipeline in Figure 3.4 uses the model only where it adds value and checks everything it returns. It runs in n8n, an automation platform that connects services through configurable nodes (n8n GmbH, n.d.).

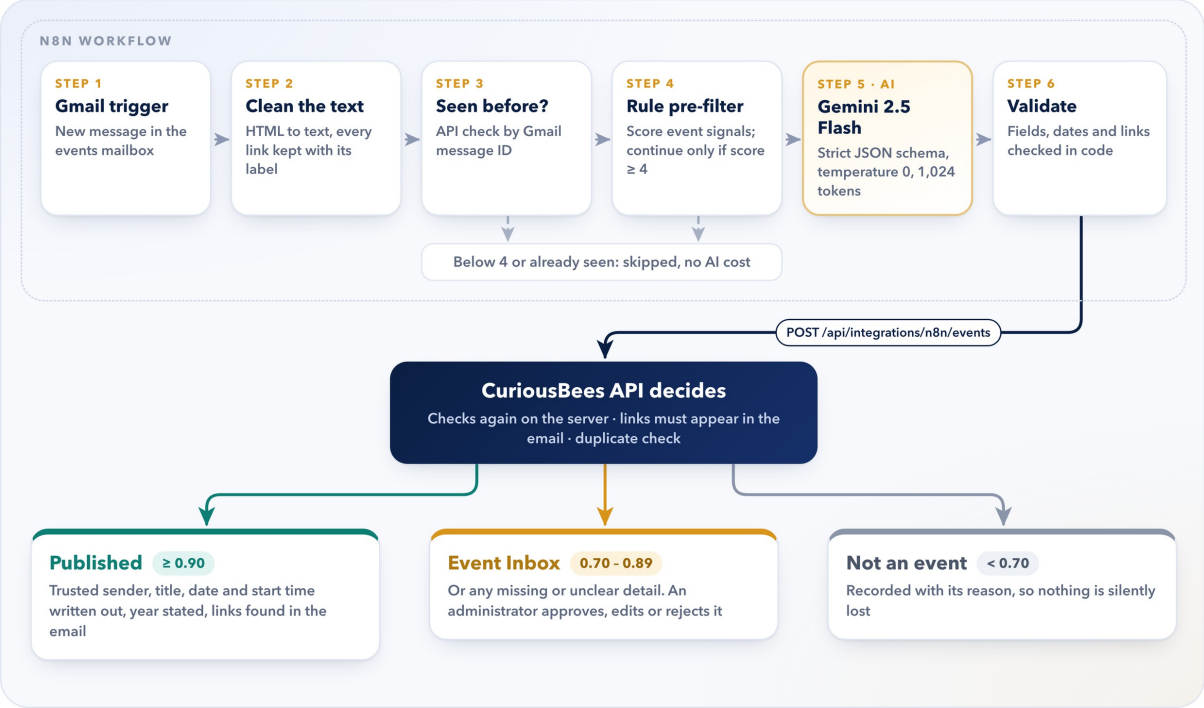


Figure 3.4: Email-to-event pipeline: rules first, the model second, and the API makes the final decision

Before the model sees anything, each email is converted to plain text with every link kept beside its label, and the API is asked whether that Gmail message ID has been seen before, so the same email is never processed twice. A rule-based pre-filter then scores the message. It adds points for event words, dates, times, venues, meeting links, registration forms and trusted senders, and takes points away for invoices, promotions, newsletters and no-reply addresses. Password resets, one-time codes, bounces and automatic replies are always rejected. Only messages with a score of at least 4 go any further, so most emails never cost anything.

The request to Gemini 2.5 Flash asks for a strict JSON schema, with the temperature at 0, the output capped at 1,024 tokens and the model's extra reasoning tokens switched off (Google AI for Developers, n.d.). The schema turns the answer into data that code can check field by field, and a temperature of 0 keeps the answers as repeatable as possible. In effect, the model behaves like a component with a defined interface rather than a chat assistant.

The answer is validated in the workflow and again in the API, which makes the final call using the confidence bands in the figure. Links are accepted only if they appear in the original email, and if the model had to guess the year, the event goes to review instead of being published. A second check catches the same event arriving in a different email, or one that someone has already posted by hand. The workflow's own tests use recorded model responses, so the test suite never calls the paid service. Figure 3.5 shows the Event Inbox, where administrators

handle the uncertain cases. When I took the screenshot it had received 11 emails, filtered out 1, sent 10 to the model, published 4 automatically and was holding 2 for review.

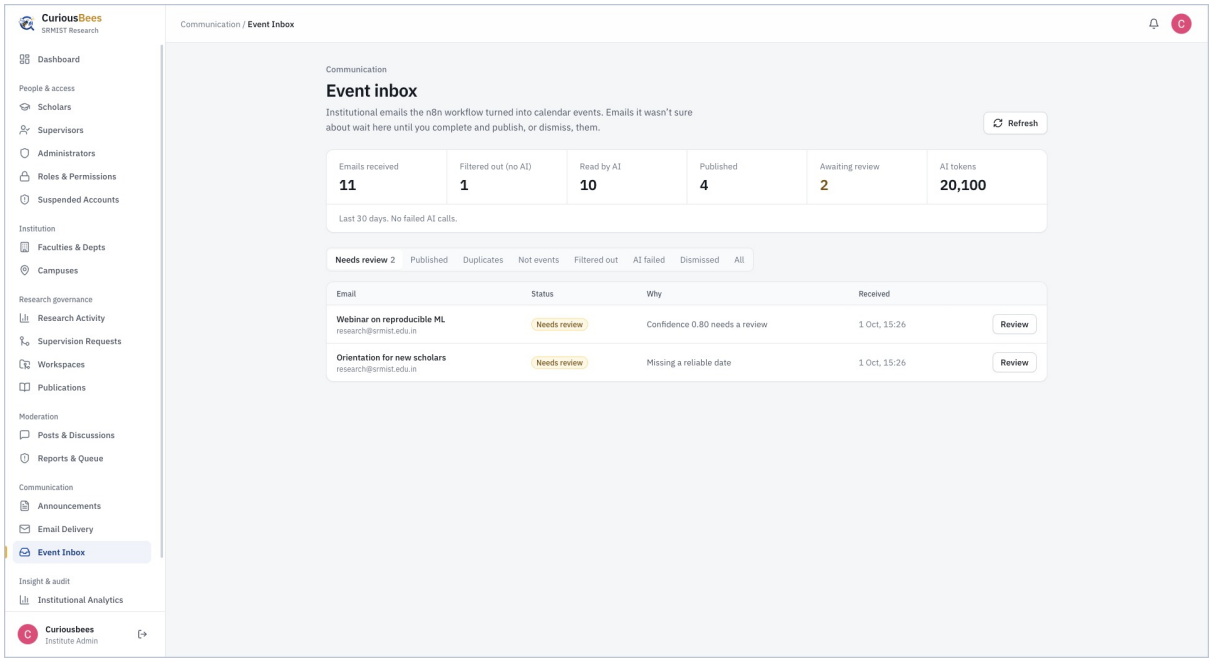


Figure 3.5: Event Inbox: emails read by the workflow, with two events waiting for an administrator

3.8 Testing, CI/CD and Cloud Deployment

Automated tests turn expected behaviour into code that runs on every change. CuriousBees has four suites (Table 3.4). Unit tests check one service or guard at a time with the database replaced by a stand-in, and finish in seconds. Integration tests run the real HTTP pipeline against a real PostgreSQL database and an S3-compatible emulator, which catches problems that only appear when the parts work together. The test framework is Jest (OpenJS Foundation, n.d.).

Table 3.4: Automated Test Suites at Delivery

Suite	Tests	What it verifies
API unit tests (Jest)	172	Services, guards, validation and decision logic, with the database replaced
API integration tests (Jest, Supertest)	61	The real HTTP pipeline against PostgreSQL and an S3 emulator
n8n workflow tests (Node.js test runner)	34	Pre-filter, request building and the workflow file, with network access blocked
Web helper tests (Node.js test runner)	3	Safe rendering of links and time zone labels
Total	**270**	**All passing at delivery**

Because the integration suite wipes and recreates its database, it must never reach a real one. It runs only against a dedicated, disposable test database on the local machine, strips database and service credentials from its environment, and blocks every network connection that leaves the machine. I added these safeguards the hard way, as Section 4.4 explains.

Continuous integration means that every change is built and tested automatically; continuous delivery means that a tested change can be released through a repeatable process (Humble & Farley, 2010). Figure 3.6 shows the pipeline I set up in GitHub Actions (GitHub, n.d.) and the AWS architecture prepared for production. The application runs in Docker containers, which behave the same on a laptop and in the cloud (Docker Inc., n.d.). The pipeline holds no long-lived cloud keys: GitHub proves its identity to AWS through OIDC and receives a short-lived role. Smaller controls keep the platform responsive under load, including pagination, rate limiting, a cap on database connections per container, indexes on the largest tables, separate liveness and readiness checks, graceful shutdown and response compression. The AWS resources themselves are created only at deployment time, which ITKM will carry out.

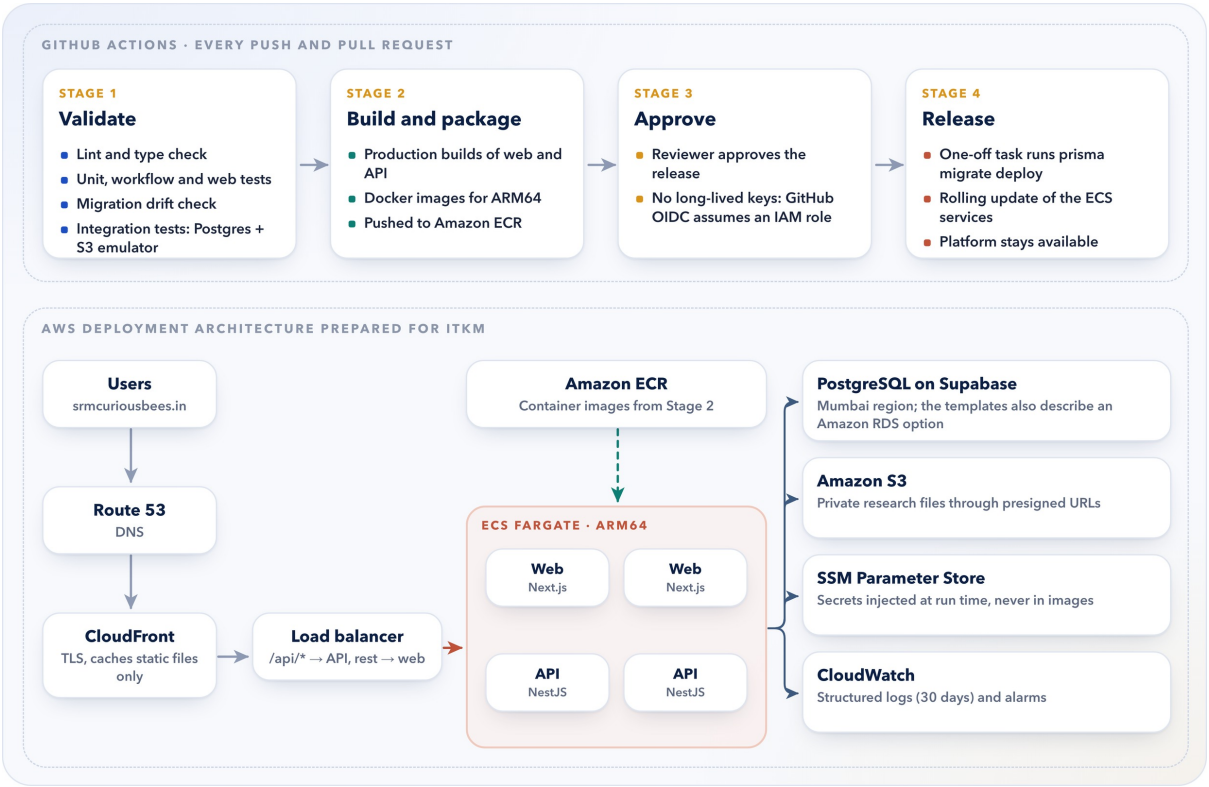


Figure 3.6: CI/CD pipeline in GitHub Actions and the AWS deployment architecture prepared for ITKM

4. DAILY/WEEKLY LOG OF ACTIVITIES

The logs in this chapter are based on the version-control history of the delivered repository. Between 25 May and 25 August 2026, 111 commits were made on 24 working days, and by delivery on 4 October the total had reached 129. Figure 4.1 shows how unevenly that work was spread, and Figure 4.2 places the main phases on a timeline. The quiet weeks in July were the weeks in which my teammates built their modules in their own repositories, which do not appear in these counts.



Figure 4.1: Weekly commits and commit types in the delivery repository, May to October 2026



Figure 4.2: Timeline of the main project phases

4.1 Weekly Log

Table 4.1 summarises each week of the internship, and Table 4.2 covers the completion and delivery work that followed.

Table 4.1: Weekly Log of Internship Activities

Week	Period (2026)	Activities
1	25 – 31 May	Internship began on 25 May. Created the monorepo (Next.js, NestJS, shared packages) as the SRM ReCollab Phase 1 MVP; first Vercel deployment after fixes to serverless handling, Prisma targets, routing and CORS; Supabase schema and seed; Firebase sign-in; first AI event prototype. (17 commits)
2	1 – 7 Jun	Workspaces with approval workflows; role-based route protection and portal dashboards; opportunities portal; admin dashboard with moderation and audit logs; modular restructure; security hardening, API documentation, environment validation and indexes; Redis and BullMQ tried and removed; sign-in moved to Clerk; API prepared for Railway. (44 commits)
3	8 – 14 Jun	Clerk sign-in with two-step verification and password reset; institute admin panel; public pages; configurable allowed email domains; user management consolidated; brand palette; research feed with discovery, nested comments and thread actions. (18 commits)
4 – 5	15 – 28 Jun	Faculty field in the feed; post management; saving and commenting; public post view with sharing; feed sorting, saved-posts filter and peer connections. (6 commits)
6 – 11	29 Jun – 9 Aug	Team modules built in the members' own repositories. One fix in the delivery repository: the API's network binding for deployment (13 Jul). (1 commit)
12	10 – 16 Aug	Events and opportunities dashboards rebuilt as modules; event editing; Curious Nexus; researcher directory with follow; scholar onboarding with email alerts and audit logging; multi-level comments; collaboration requests; chat module. (14 commits)
13	17 – 23 Aug	Authentication moved from Clerk to Supabase; email OTP fallback with rate limiting; integrations for Google Workspace and Zoom; supervisor request management; scholar reassignment; unified account settings; capacity standardised at six. (10 commits)
14	24 – 25 Aug	Socket-based chat removed in favour of a simpler design; research workflow documentation and capacity analysis; internship completed on 25 August. (1 commit)

Table 4.2: Completion and Delivery after the Internship Period

Period (2026)	Activities	Outcome
8 Sep	README overhaul, repository clean-up and removal of legacy names	Clean, documented repository (2 commits)
28 Sep – 4 Oct	Phase 2 institutional hierarchy (v0.4.0) with tests; AWS infrastructure, IAM policies and CI/CD; ResearchParticipantGuard, admin safety rules and root-admin protection; redesign and design system; single and bulk provisioning; Chat spaces; workspace creation; email-to-event pipeline with tests; final build fix	Integrated, tested platform delivered on 4 October (16 commits)
October	Production hardening: per-session rate limits, a time limit on sign-in checks, dependency updates, a safe load-testing harness and stronger database guards; development data moved to the Supabase cloud database	Better prepared for deployment by ITKM

4.2 Daily Log

Table 4.3 lists every day on which work was committed to the delivery repository during the internship. Work done in my teammates' separate repositories is not shown here.

Table 4.3: Daily Log of Internship Activities

Date	Activity
Mon 25 May	Internship began under the department's Faculty-Guided Summer Internship Programme.
Fri 29 May	Created the monorepo as the SRM ReCollab Phase 1 MVP; rebranded to ReCollab; first Vercel deployment with a health check, production CORS, a serverless handler and Prisma binary targets.
Sat 30 May	App-specific Vercel settings; health check and global prefix refined; Supabase schema, seed script and Prisma direct URL.
Sun 31 May	Firebase sign-in; AI event ingestion prototype; event notifications; analytics and search services.
Wed 3 Jun	Workspace management with approvals; role-based sign-in and route protection; portal layout and role pages.
Thu 4 Jun	Modular restructure; opportunities portal; admin dashboard with moderation, audit logs and monitoring; onboarding guard.
Fri 5 Jun	Security hardening, API documentation, environment validation and indexes; dead code and redundant tables removed.
Sat 6 Jun	Central environment configuration and a diagnostic "doctor" script; logging middleware; standard health checks.
Sun 7 Jun	Sign-in moved to Clerk; faculty–department validation and supervisor approval rules; build orchestration; Railway hardening; Redis and BullMQ removed.
Mon 8 Jun	Clerk sign-in with 2FA and password reset; institute admin panel; route-group conflict fixed; public pages; allowed email domains.
Tue 9 Jun	User management consolidated; brand colour palette; session synchronisation fixes.
Thu 11 Jun	Scholar portal routing, thread details and workspace pages.

Date	Activity
Fri 12 Jun	Research feed with discovery endpoints, nested comments and thread actions.
Sun 21 Jun	Faculty field in the feed; chat and post-management modals; saving and comments.
Tue 23 Jun	Public post view with a share modal.
Wed 24 Jun	Feed sorting, saved-posts filter and peer connections with notifications.
Mon 13 Jul	API network binding corrected so the server listens on the default interfaces.
Mon 10 Aug	Events and opportunities dashboards rebuilt as modules with role-based access.
Tue 11 Aug	Event editing; Curious Nexus; researcher directory with follow; scholar onboarding with email alerts and audit logging.
Wed 12 Aug	Profile menu, feed navigation and access features across dashboard, events and community.
Thu 13 Aug	Multi-level comments with notifications; richer opportunity model; collaboration flow and chat interface.
Fri 14 Aug	Chat module with a gateway; deployment documentation.
Thu 20 Aug	Sign-in moved from Clerk to Supabase; email OTP fallback; Google Workspace and Zoom integrations; supervisor requests.
Fri 21 Aug	Scholar reassignment; unified account settings; capacity standardised at six.
Tue 25 Aug	Socket chat removed; workflow documentation and capacity analysis; internship completed.

4.3 Major Projects and Their Outcomes

By delivery, the internship had produced a working, integrated platform. The outcomes fall into five groups:

- three portals, for scholars, supervisors and administrators, with every permission enforced by the API;
- a data model that mirrors the institution, from campus to faculty to department;
- private collaboration through workspaces with direct-to-storage files, the research feed and Curious Nexus;
- AI that saves manual work without publishing guesses; and
- deployment readiness, with 270 automated tests, a CI/CD pipeline, container images and a prepared cloud design.

The screenshots that follow were taken from the running application in October 2026. I signed in as each of the three roles in turn, with the platform connected to its cloud database, so what you see is the data the platform actually holds rather than mock-ups. A visitor first reaches the public landing page (Figure 4.3). Signing in is deliberately simple: there is a single Google button, and only accounts on approved SRMIST domains that an administrator has already created can get in (Figure 4.4).

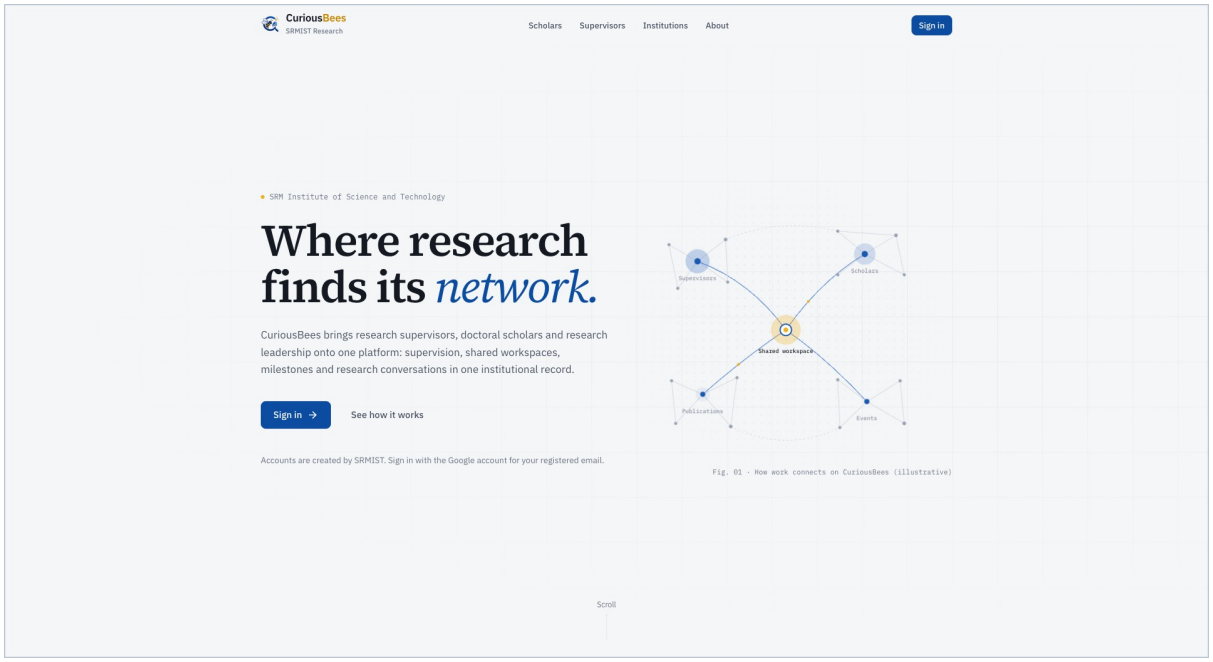


Figure 4.3: Public landing page of CuriousBees

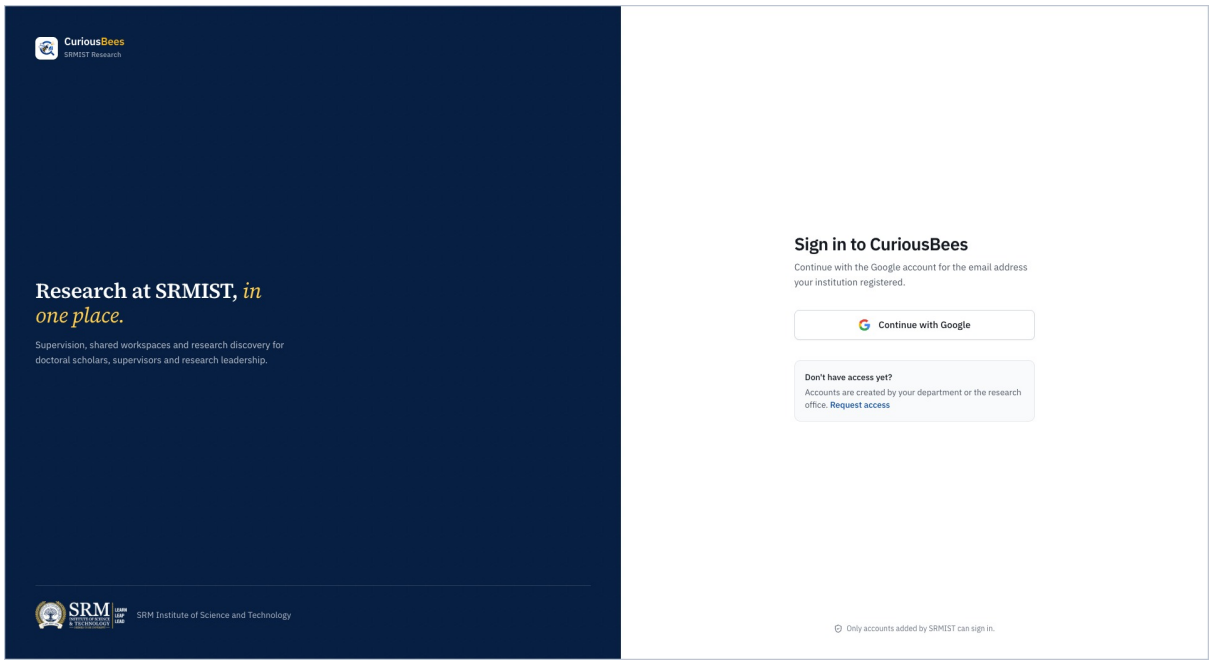


Figure 4.4: Sign-in page: Google accounts on approved SRMIST domains only

A supervisor lands on an overview of their scholars, their profile and researchers they may want to know (Figure 4.5). The research feed (Figure 4.6) is shared by scholars and supervisors across the institution, with filters for papers, questions and updates.

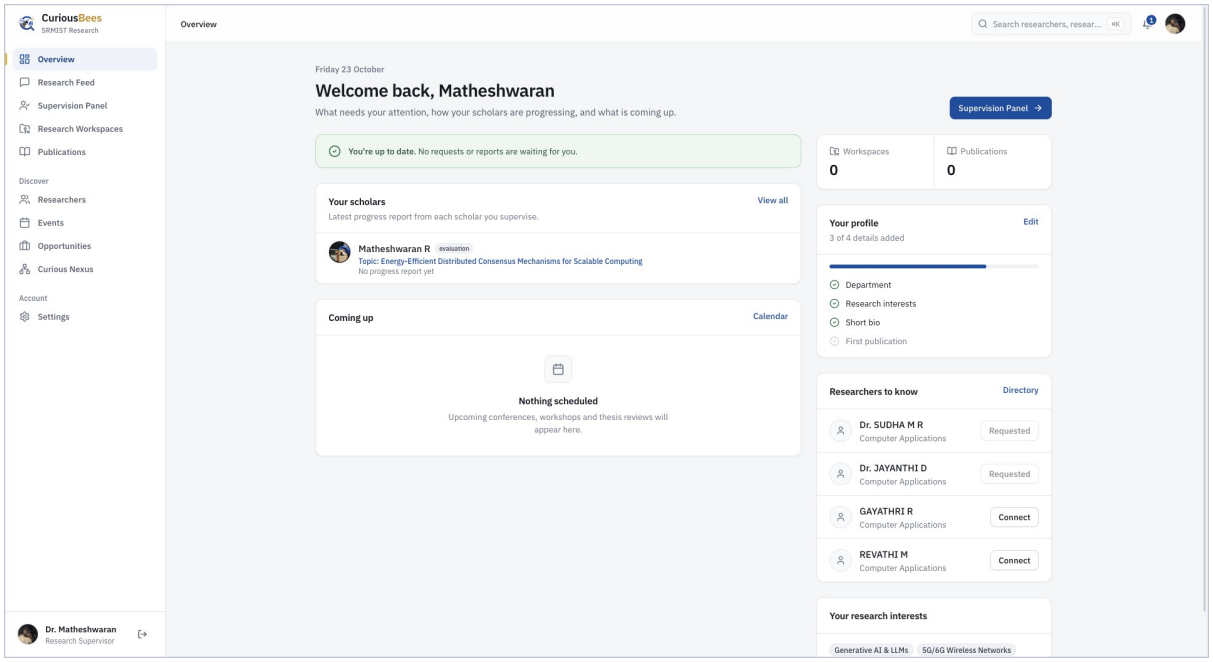


Figure 4.5: Supervisor overview with assigned scholars and suggested researchers

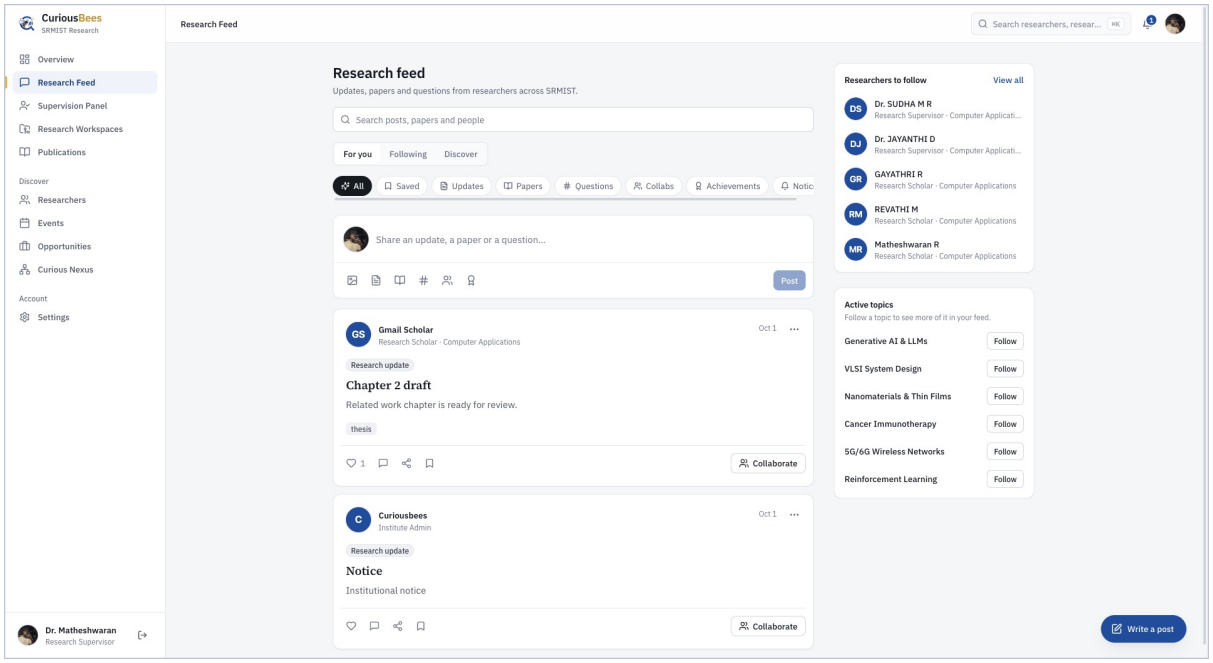


Figure 4.6: Research feed with posts, filters and researchers to follow

A scholar keeps their research record on the My Research page, which shows the thesis topic, the current stage and the supervisor (Figure 4.7). The supervisor follows the same scholars from the Supervision Panel (Figure 4.8) and can open a research workspace for them (Figure 4.9).

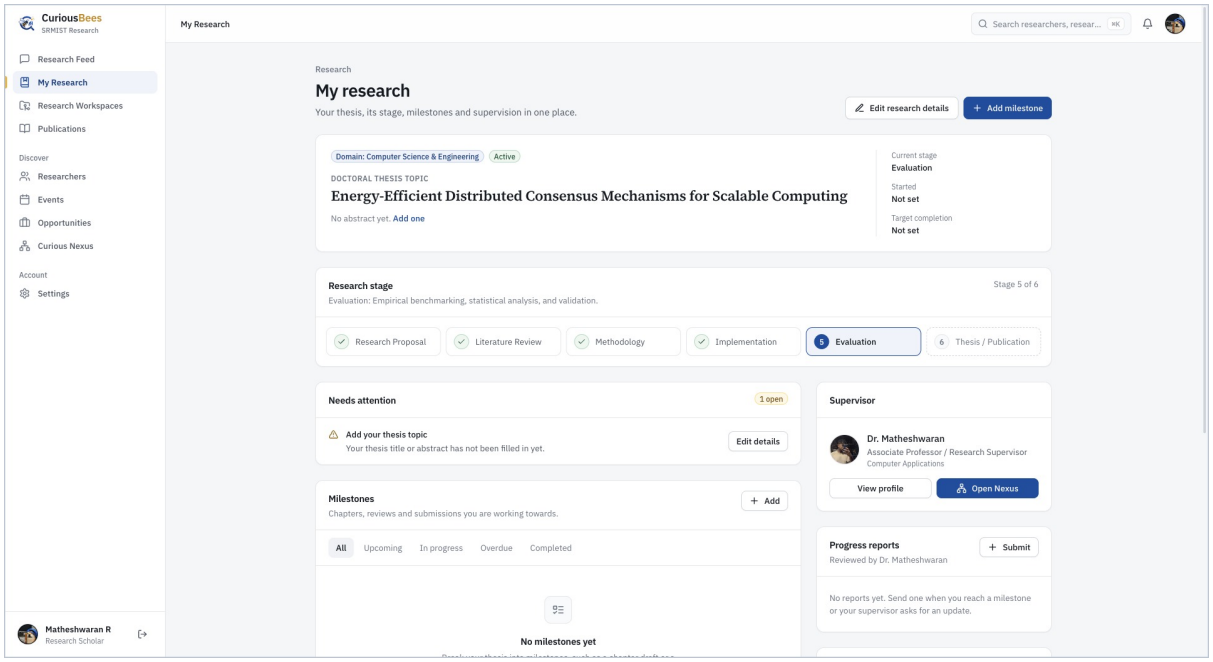


Figure 4.7: My Research page of a scholar: thesis topic, research stage and supervisor

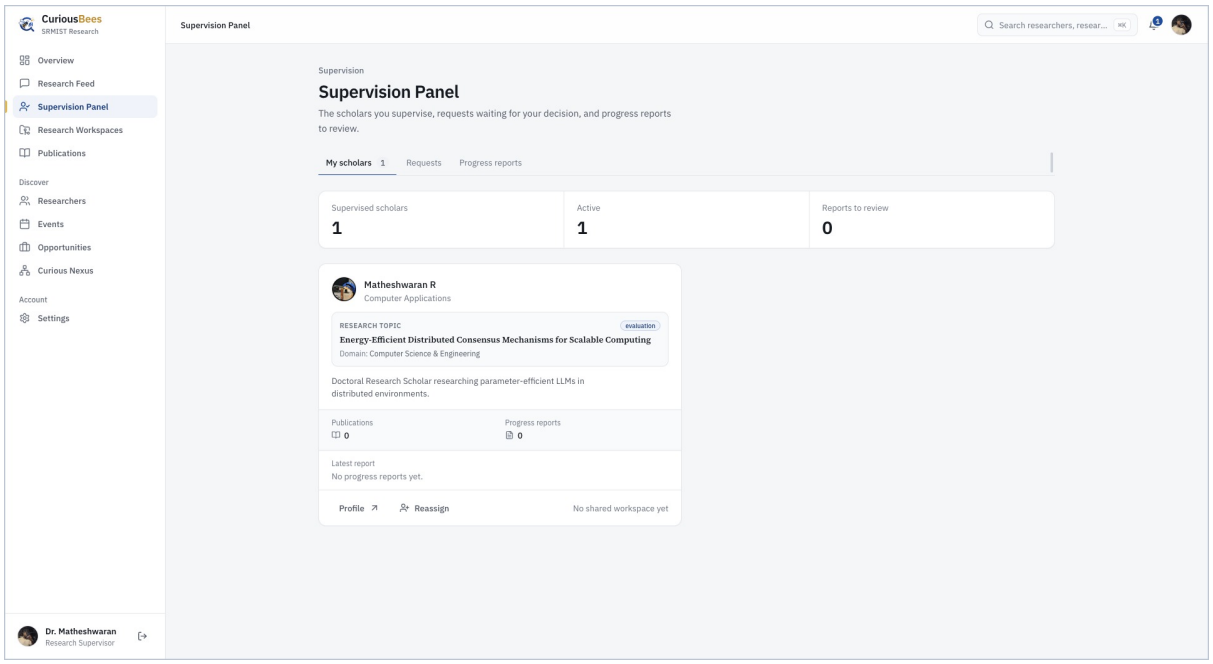


Figure 4.8: Supervision Panel listing a supervisor's scholars and their reports

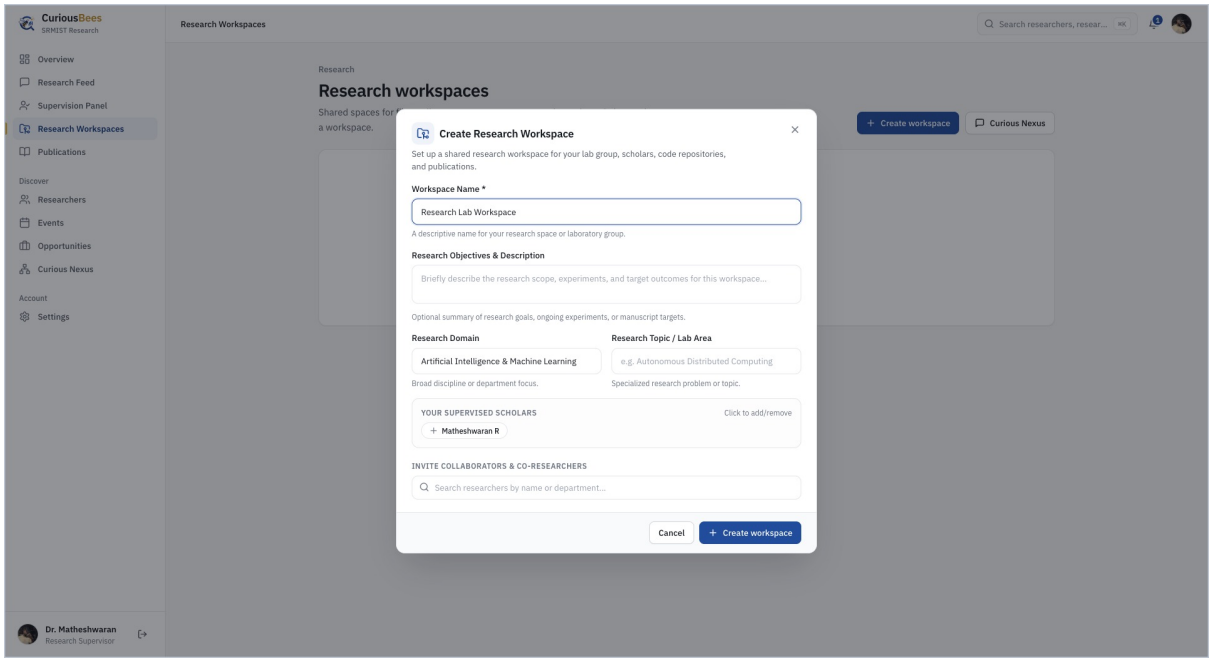


Figure 4.9: Creating a research workspace

Figure 4.10 is my favourite screenshot in this report. The International Research Symposium on the calendar was never typed in by anyone: the workflow read it from an institutional email, and the event page says so.

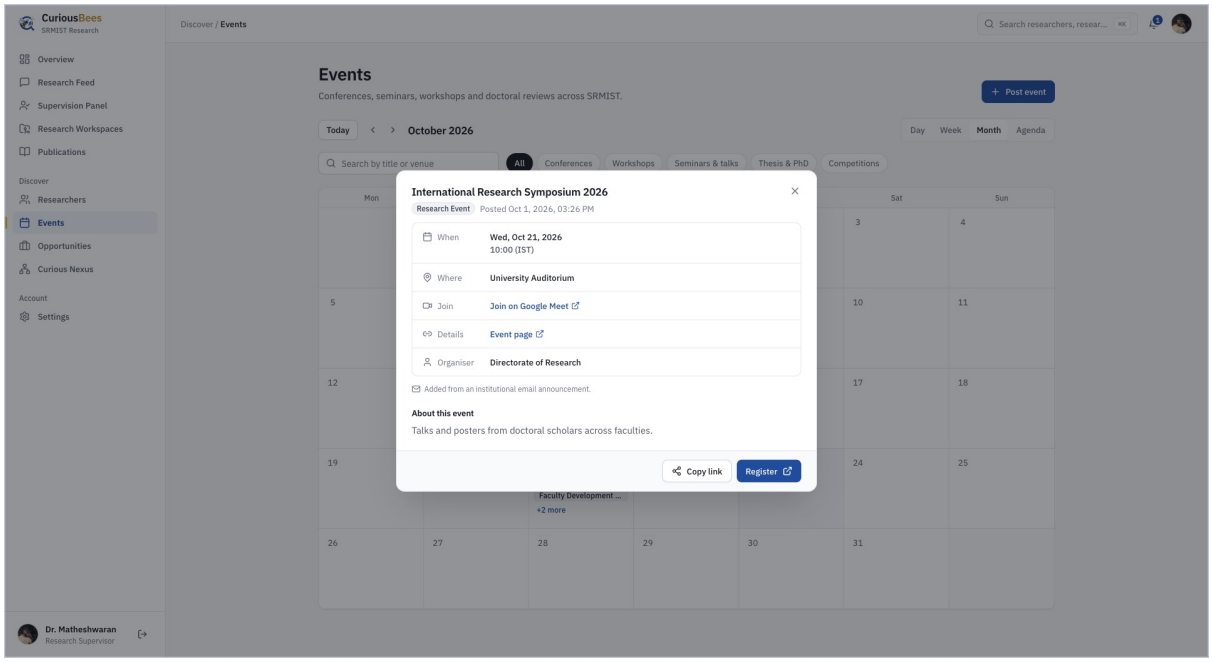


Figure 4.10: Events calendar with an event added automatically from an institutional email

Administrators work in a separate console. They create accounts one at a time or in bulk from a spreadsheet (Figure 4.11) and maintain the institutional structure of 11 faculties and 29 departments (Figure 4.12). Every sensitive action they take, and every event the workflow publishes, is recorded in the audit log.

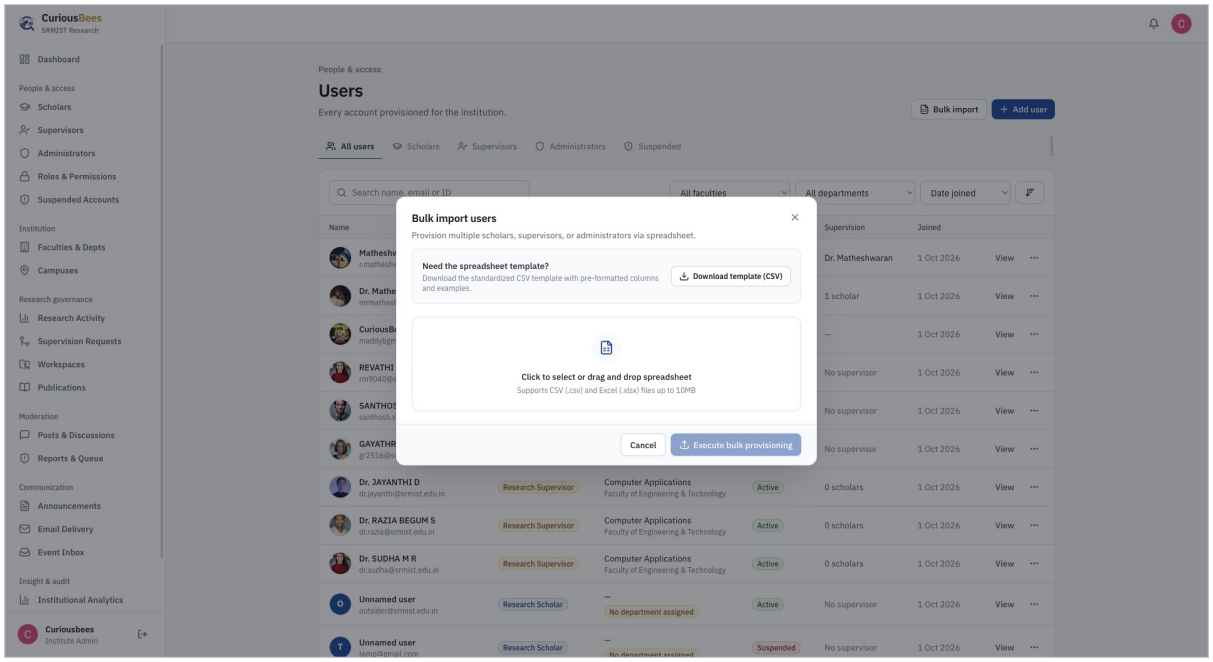


Figure 4.11: User directory with bulk import from a spreadsheet

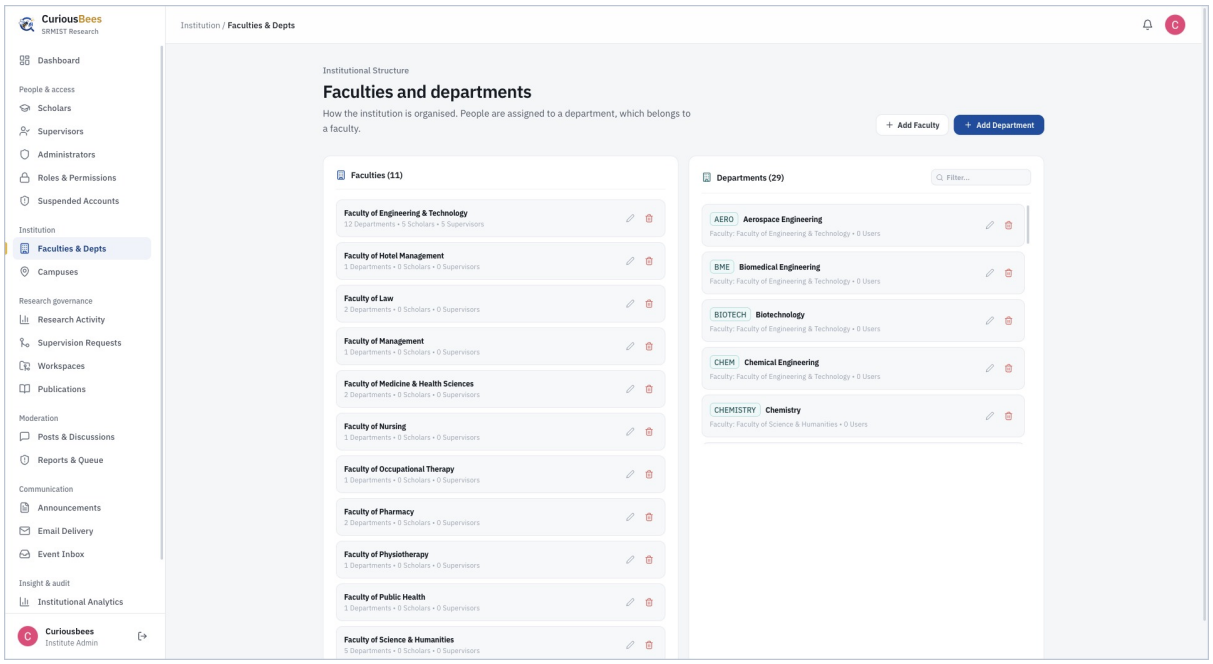


Figure 4.12: Faculties and departments managed by the administrator

These outcomes are backed by automated evidence. At delivery all 270 tests passed, and the production builds of both applications completed successfully.

4.4 Challenges Faced and How They Were Addressed

Not everything went smoothly. Table 4.4 lists the main problems we ran into and how each one was handled.

Table 4.4: Challenges Faced and Solutions

Challenge	How it was addressed
Deploying the NestJS and Prisma monorepo as serverless functions failed repeatedly in the first week.	Eight deployment fixes over 29 and 30 May: a serverless handler, Prisma binary targets, per-app settings, a health check and the API prefix. The experience led to long-running containers later.
The authentication provider changed twice, from Firebase to Clerk and from Clerk to Supabase.	The provider now only proves identity, while CuriousBees decides permissions from its own database, which made the final migration manageable.
A development shortcut that bypassed sign-in and CORS would have been dangerous in production.	Replaced with administrator-created accounts, strict production configuration checks and server-side role guards.
Route conflicts and small coding errors broke production builds.	Fixed the conflicting route and the hook order; the CI pipeline now runs linting, type checks and builds on every change.
Redis queues and a socket-based chat server added services to deploy and watch.	Removed both after measuring the real need. Scheduled work uses a PostgreSQL advisory lock, and conversations moved to Curious Nexus and Chat spaces.
Team modules were built in separate repositories and had to become one product.	Used the shared type packages as the contract, adapted each contribution to the common structure and access rules, and verified the result with integration tests.
During one test run, a configuration fallback pointed the integration suite at my local development database, and its demonstration data was lost.	Treated as a design flaw: tests never read the development configuration, the reset fails closed and runs only against one disposable test database, credentials are stripped and outbound network access is blocked. In October I added a live check of the database's identity before any reset.
A language model can misread dates or invent links, and calling it for every email would be costly.	A rule-based filter before the model, a strict JSON schema at temperature 0, independent validation and human review for uncertain cases.

The testing incident is the one I think about most. Nothing reached production, and what was lost was only demonstration data, but it showed me that a safety rule which depends on people remembering it is not really a safety rule. Since then I have tried to make the dangerous path impossible rather than merely discouraged.

5. SKILLS ACQUIRED

5.1 Technical Skills

Figure 5.1 maps the skills I built during the internship. I have grouped them around the four areas I now want to work in, namely full stack development, cloud, DevOps, and AI and automation, together with leadership and quality. Table 5.1 relates each area to how I developed it and where it applies.



Figure 5.1: Mind map of the skills developed during the internship

Table 5.1: Technical Skills and Their Application

Area	How I developed it	Where it applies
Full stack development	Building the web application, the 253-endpoint API and the shared packages	Any modern web product
Data modelling	A 54-model schema and the Phase 2 hierarchy migration	Institutional and enterprise systems
Cloud	Presigned S3 storage, Supabase, and the AWS design with IAM and OIDC	Cloud engineering roles
DevOps	GitHub Actions, Docker images, forward-only migrations, health checks	DevOps and platform teams
AI and automation	The n8n and Gemini email-to-event pipeline with validation and review	Document processing and workflow automation
Testing and security	270 automated tests, OWASP-based hardening, test isolation	Any professional software team

The biggest change, though, was not any single technology. I learned to think about the system as a whole: how a choice in the database schema affects authorization, how the authentication provider affects deployment, and how a single missing configuration value can make a system unsafe.

5.2 Soft Skills

Leading eleven people across six tracks taught me to set a direction, break a large goal into pieces that people could own, and still take responsibility for the combined result. I had to explain interfaces and conventions to teammates, report progress to the Faculty Coordinator and write documentation for readers outside the project, and each of these needed a different way of communicating.

Integration also meant disagreement. Two teammates would sometimes solve the same problem in different ways, and both would have a point. Conflicts went better when I referred to the shared contracts and the institution's requirements rather than to personal preference, and when I explained my reasoning instead of just the decision. Time management turned out to be mostly about cutting: delivering on time meant dropping work that added complexity without adding value. And when the testing incident happened, I reported it openly and fixed its cause rather than its symptom, which I think mattered more than the fix itself.

5.3 New Knowledge and Competencies Developed

Some of what I learned is rarely covered in coursework. I learned to design for operational safety, with fail-closed checks, forward-only migrations and tests isolated from real data. I learned how to put a language model into a production workflow with structured output, rule-based filtering, independent validation and human review. I learned to plan cloud infrastructure with least-privilege access and keyless deployment, and to build the governance features an institution needs, such as administrator-created accounts, a protected root administrator and audit trails. Finally, I learned to write documentation that lets another team run the system.

5.4 Analysis and Discussion

Looking back, the decisions that helped most were structural rather than clever. The monorepo and the shared types kept the frontend and the backend in step while many people worked at once. Keeping authorization inside our own application let us change authentication providers twice without touching the access rules. And the automated tests gave me confidence every time I merged a teammate's work.

5.4.1 Achievement of Objectives

Table 5.2 relates the objectives from Section 1.2 to the work through which they were met.

Table 5.2: Objectives and How They Were Achieved

Objectives	Achieved through
1–2 A usable platform; how web applications fit together	The monorepo, web application and API built from week 1 and delivered on 4 October (Sections 3.4–3.5)
3–4 Institutional model and roles; service integrations	Role guards, the Phase 2 hierarchy, and the storage, email and Google Workspace integrations
5 Responsible use of AI	The email-to-event pipeline with confidence bands and the Event Inbox (Section 3.7)
6–7 Security, testing and deployment	OWASP-based hardening, 270 tests and the CI/CD pipeline with the AWS design (Sections 3.6 and 3.8)
8 Leadership and documentation	Planning, review and integration throughout; 40 documents for the ITKM handover

5.4.2 What I Would Do Differently

I would choose the authentication provider after a short evaluation at the start instead of changing it twice. I would add infrastructure such as message queues only when there is a measured need. And I would put automated tests and test isolation in place in the first week rather than the last; the incident in Section 4.4 would never have happened if the destructive reset had been built to fail closed from the beginning. Since production deployment rests with ITKM, I have not yet run the platform under real institutional load, so full-scale load testing, production monitoring and incident response are the areas I most want to learn next.

6. CONCLUSION

6.1 Summary of the Internship Experience

The three-month in-house internship at the Department of Computer Applications, from 25 May to 25 August 2026, was the most practical learning experience of my studies so far. As Team Lead and Full Stack Developer of the eleven-member CuriousBees team, I helped turn an empty repository into a research collaboration and governance platform with separate portals for research scholars, supervisors and institute administrators. It covers supervision, research records, private workspaces, a research feed, opportunities, events, notifications and a full administrative console, along with an AI pipeline that turns event emails into calendar entries.

Technically, the work spanned a Next.js and React frontend, a NestJS API with 253 endpoints, a 54-model PostgreSQL schema on Supabase, integrations with Amazon S3, Brevo and Google Workspace, an n8n workflow with Gemini 2.5 Flash, 270 automated tests and a CI/CD pipeline with a prepared AWS architecture. As Team Lead I planned the work, divided it among the tracks, reviewed and integrated my teammates' contributions, and delivered the final build on 4 October 2026 for deployment by ITKM.

6.2 Influence of the Internship on Career Goals

Before this internship I thought of software mainly as code. CuriousBees showed me the distance between code that works on my laptop and a system an institution can depend on, and that distance is where I want to build my career.

My direction is now clear. I want to work as a **Full Stack Developer** who is just as comfortable with **Cloud** and **DevOps**, and who can bring **AI and Automation** into products responsibly. Full stack development gave me the ability to build a feature from the database to the screen. Cloud and DevOps work taught me how to ship it and run it safely. The email-to-event pipeline convinced me that AI and automation are most useful when they sit inside careful engineering, with rules, validation and a human in the loop. Next, I want to deepen my knowledge of AWS, gain experience operating systems in production, and keep building automation that combines workflows with language models.

I am grateful to the Department of Computer Applications for this opportunity, and I look forward to carrying what I learned into my final semester and beyond.

REFERENCES

Amazon Web Services. (n.d.). *AWS documentation: Amazon S3, Amazon ECS and Amazon CloudFront*. https://docs.aws.amazon.com/

Brevo. (n.d.). *Brevo API reference*. https://developers.brevo.com/

Docker Inc. (n.d.). *Docker documentation*. https://docs.docker.com/

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Doctoral dissertation, University of California, Irvine].

GitHub. (n.d.). *GitHub Actions documentation*. https://docs.github.com/en/actions

Google AI for Developers. (n.d.). *Gemini API: Structured output*. https://ai.google.dev/gemini-api/docs/structured-output

Google for Developers. (n.d.). *Google Workspace developer documentation*. https://developers.google.com/workspace

Humble, J., & Farley, D. (2010). *Continuous delivery: Reliable software releases through build, test, and deployment automation*. Addison-Wesley.

Kleppmann, M. (2017). *Designing data-intensive applications*. O'Reilly Media.

Meta Open Source. (n.d.). *React documentation*. https://react.dev/

Microsoft. (n.d.). *TypeScript documentation*. https://www.typescriptlang.org/docs/

n8n GmbH. (n.d.). *n8n documentation*. https://docs.n8n.io/

NestJS. (n.d.). *NestJS documentation*. https://docs.nestjs.com/

OpenJS Foundation. (n.d.). *Jest documentation*. https://jestjs.io/docs/getting-started

OWASP Foundation. (2021). *OWASP Top 10:2021*. https://owasp.org/Top10/

PostgreSQL Global Development Group. (n.d.). *PostgreSQL documentation*. https://www.postgresql.org/docs/

Prisma Data. (n.d.). *Prisma ORM documentation*. https://www.prisma.io/docs

SRM Institute of Science and Technology. (n.d.). *Department of Computer Applications*. https://www.srmist.edu.in/department/department-of-computer-applications/

SRM Institute of Science and Technology. (2023). *Strategic plan 2023 (Plan period 2024–2033)*. https://webstor.srmist.edu.in/web_assets/downloads/2023/srmist-strategic-plan.pdf

Supabase. (n.d.). *Supabase Auth documentation*. https://supabase.com/docs/guides/auth

Tailwind Labs. (n.d.). *Tailwind CSS documentation*. https://tailwindcss.com/docs

Vercel. (n.d.). *Next.js documentation*. https://nextjs.org/docs

APPENDIX A: GLOSSARY OF TERMS

Table A.1 explains the technical terms and abbreviations used in this report.

Table A.1: Glossary of Terms

Term	Meaning
API	Application Programming Interface; the HTTP endpoints through which the web application talks to the server.
REST	Representational State Transfer; an architectural style for APIs based on resources and HTTP methods.
Monorepo	A single repository that holds several related applications and packages.
ORM	Object-Relational Mapper; a library that maps database tables to typed objects in code (Prisma here).
Migration	A versioned script that changes the structure of a database.
RBAC	Role-Based Access Control; granting permissions to roles rather than to individual people.
Presigned URL	A short-lived, server-signed link that gives direct access to one object in private storage.
CI/CD	Continuous Integration and Continuous Delivery; automated building, testing and release of software.
OIDC	OpenID Connect; used here so the CI pipeline can get short-lived cloud credentials without stored keys.
LLM	Large Language Model; an AI model trained on text that can read and generate language.
Structured output	An LLM answer constrained to a defined schema, such as JSON with fixed fields.
Advisory lock	A PostgreSQL lock that lets only one process run a given task at a time.
OWASP	Open Worldwide Application Security Project, publisher of the OWASP Top 10 web security risks.
NAAC / UGC	National Assessment and Accreditation Council / University Grants Commission.